



SMART SHOP MANAGEMENT SYSTEM

Unit Code: BIT3208

Unit Name: Advanced Web Design and Development

Student Name: YASSIR HARUNA

Admission Number: BSCCS/2024/61491

Course: COMPUTER SCIENCE

Lecturer Name: MICHAEL NYORO

Semester/Academic Year: 3.2

Technologies Used:

Node.js, Express.js, Vanilla JavaScript, HTML5, CSS3, PHP, MySQL, JWT
Authentication, XAMPP

GitHub/Portfolio Link: <https://github.com/yassirharuna/Smart-Shopping-Management-System>

Table of Contents

Table of Contents	1
WEEK 1: Local Environment Setup	4
The Required Evidence	4
WEEK 2: Wireframes and GUI Design	8
Required Evidence	8

Smart Shop Management System | Advanced Web Design and Development

Week 2 Reflection	10
WEEK 3: JavaScript and PHP Basics	10
Required Evidence.....	10
Week 3 Reflection	14
WEEK 4: Dynamic Backend Processing & Real User Interaction	15
Task 1: Understanding Server-Side Programming (Practical)	15
Goal	17
Purpose of Task 5	20
Why Modern Backend Alternatives Matter	20
Comparison: PHP vs Modern Backend Technologies	21
Conceptual Mapping	22
PHP → Node.js	22
Sessions → JWT	22
MySQL → MongoDB.....	22
Real-World Usage Examples.....	23
Week 4 Reflection.....	23
WEEK 5: Database Components & CRUD Operations.....	23
Task 1: Introduction to Databases (Concept + Understanding).....	24
Why Databases Are Important	24
Database Types.....	24
Task 2: Database Creation (Practical)	25
Task 3: Creating Tables	25
Task 4: CRUD Operations.....	26
Task 5: Connecting PHP to Database.....	27
Task 6: Fetching Database records using PHP	29
Introduction to Modern Backend Alternatives	29
Comparison of Traditional and Modern Technologies	30
Understanding the MEVN Stack.....	31
MEVN Equivalent Example (Explanation Only).....	31
Explanation of the Code (Theory)	32
Week 5 Logbook Reflection.....	32

WEEK 1: Local Environment Setup

Installation and Testing of Local Development Environment

The Required Evidence

Fig 1: Installation of XAMPP on Windows

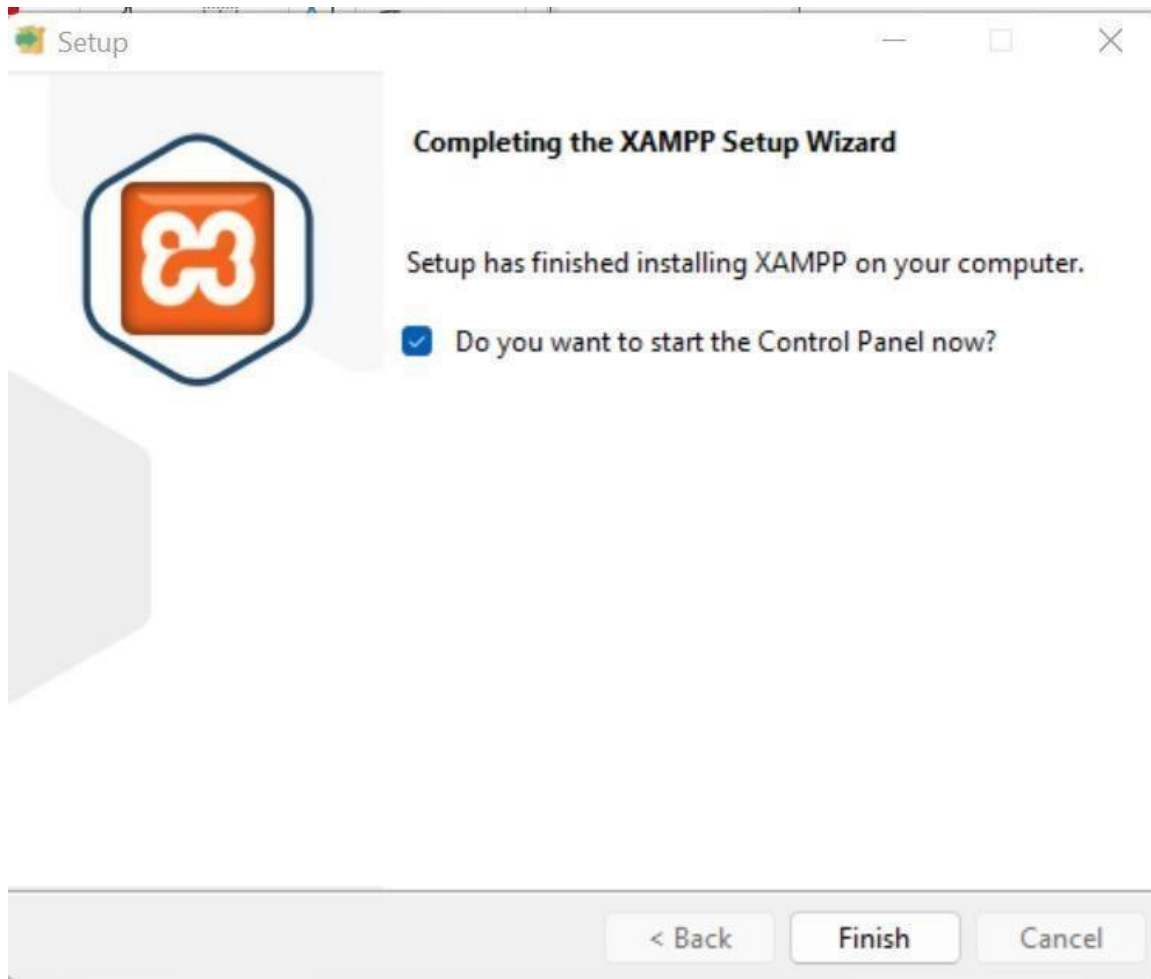


Fig 2: Apache and MySQL services running in XAMPP Control Panel

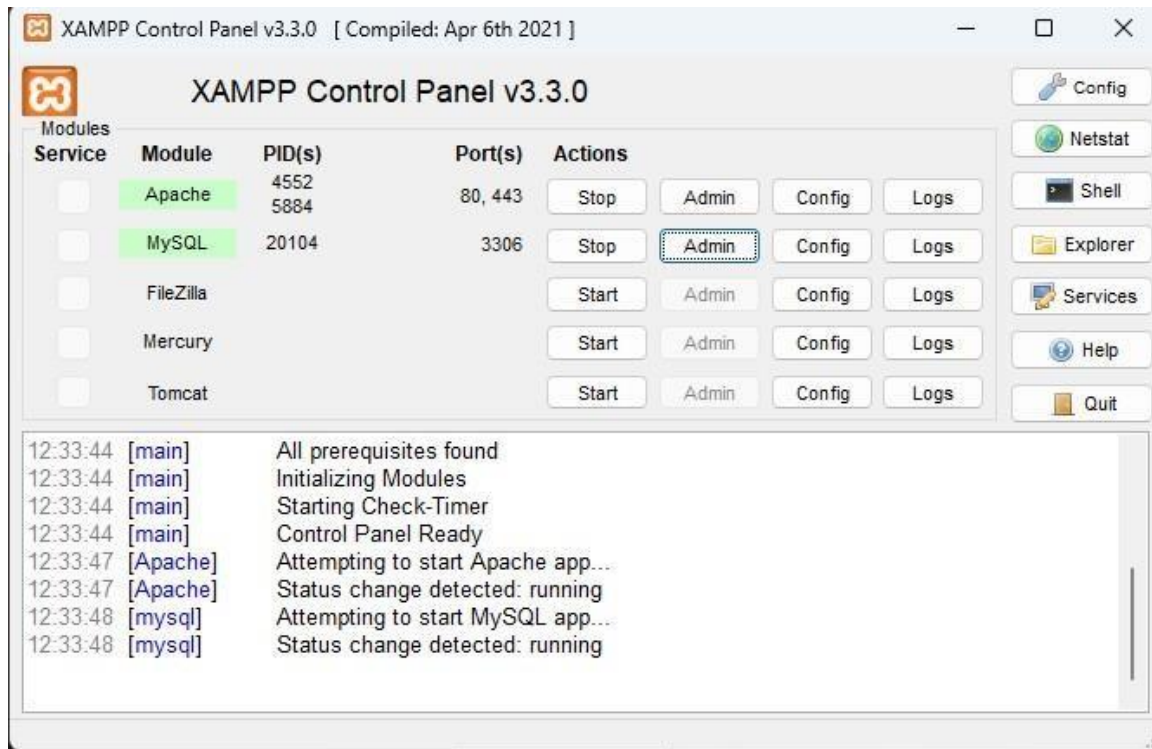


Fig 3: Localhost test page loading successfully in browser

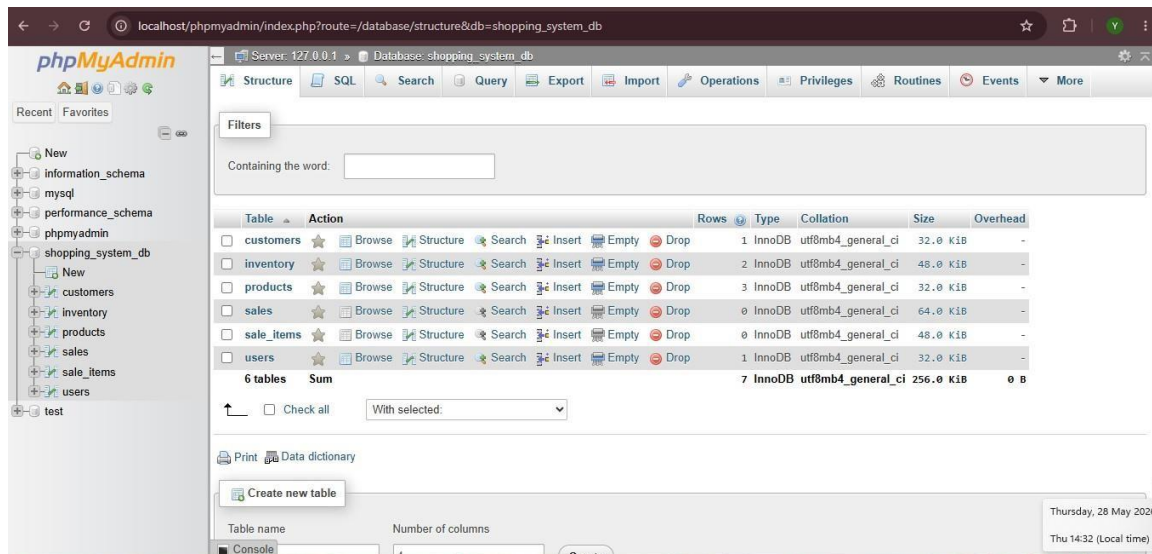


Fig 4: First PHP Hello World test page rendered in browser



Fig 5: Successful database connection test to shopping_system_db using PHP mysqli_connect()

```
api > config > db.php > ...
1  <?php
2
3
4  // Database Credentials
5  define('DB_HOST', '127.0.0.1');
6  define('DB_PORT', '3306');
7  define('DB_USER', 'root');
8  define('DB_PASS', '');
9  define('DB_NAME', 'shopping_system_db');
10
11  /**
12   * Connects to the MySQL database via PDO.
13   *
14   * @return PDO|null The PDO database connection instance, or null on error.
15   */
16  function connectDB() {
17      static $conn = null;
18
19      if ($conn !== null) {
20          return $conn;
21      }
22
23      try {
24          $dsn = "mysql:host=" . DB_HOST . ";port=" . DB_PORT . ";dbname=" . DB_NAME;
25          $options = [
26              PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
27              PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
28              PDO::ATTR_EMULATE_PREPARES => false,
29          ];
```

Week 1 Reflection

I installed XAMPP to host the PHP and MySQL components of the Smart Shop Management System. Apache and MySQL services were configured and tested successfully. I created the shopping_system_db database and confirmed connectivity using a PHP mysqli_connect() script. A Hello World PHP file verified that the server was serving PHP pages correctly. The main challenge was resolving port conflicts before Apache could start, which I fixed by adjusting the configuration.

WEEK 2: Wireframes and GUI Design

User Interface Planning and System Design

Required Evidence

Fig 1: Wireframe of the Login page in Figma

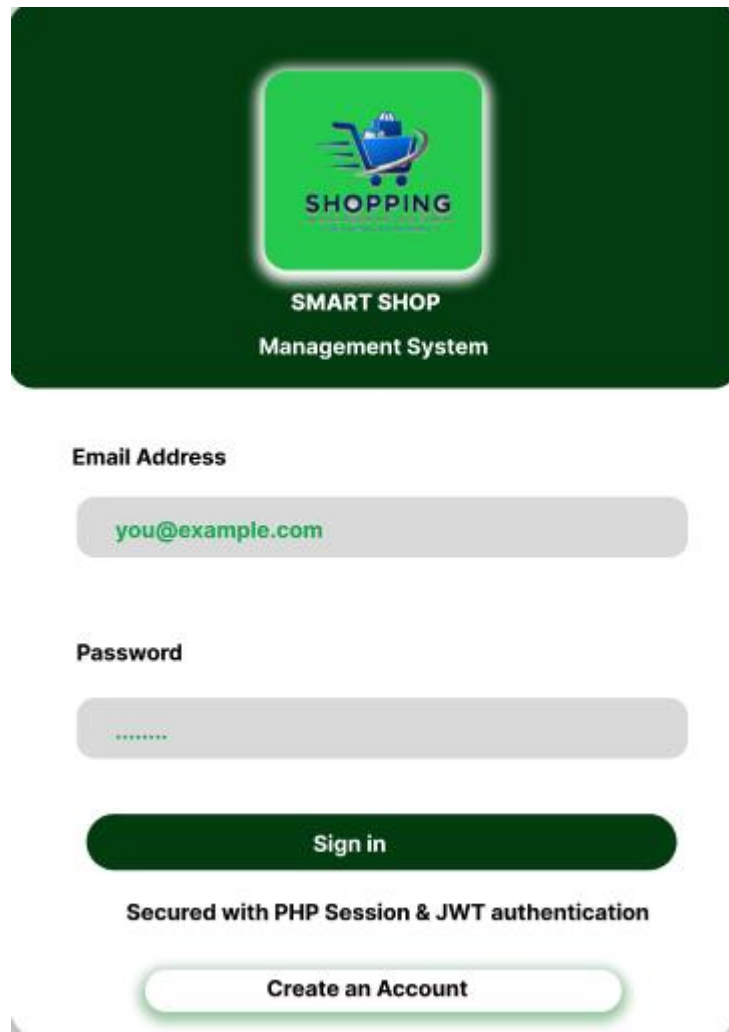


Fig 2: Dashboard layout design showing KPI cards and navigation in Figma

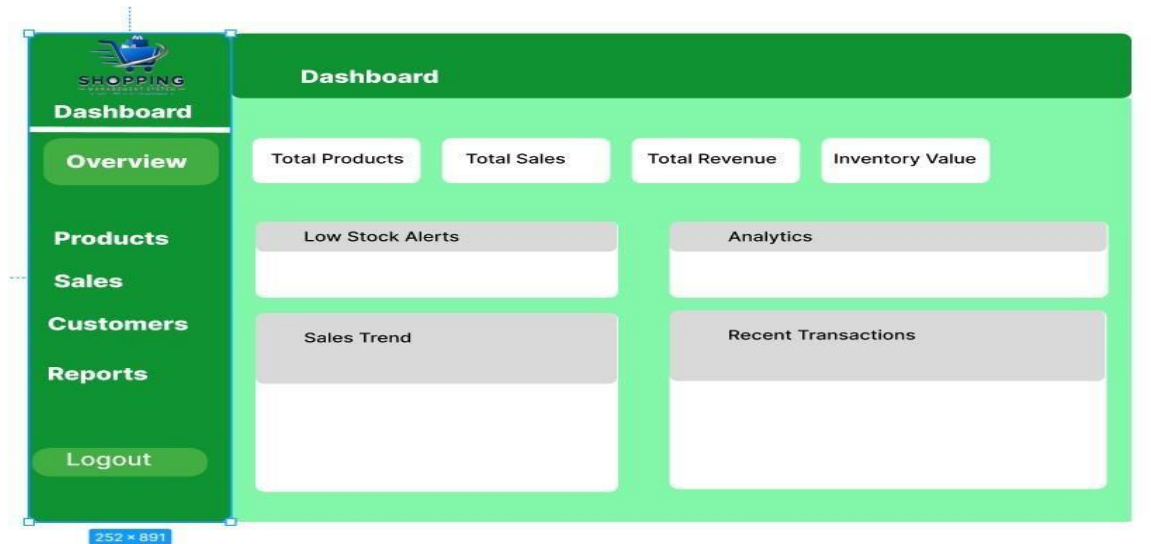


Fig 3: Mobile view mockup of the products page

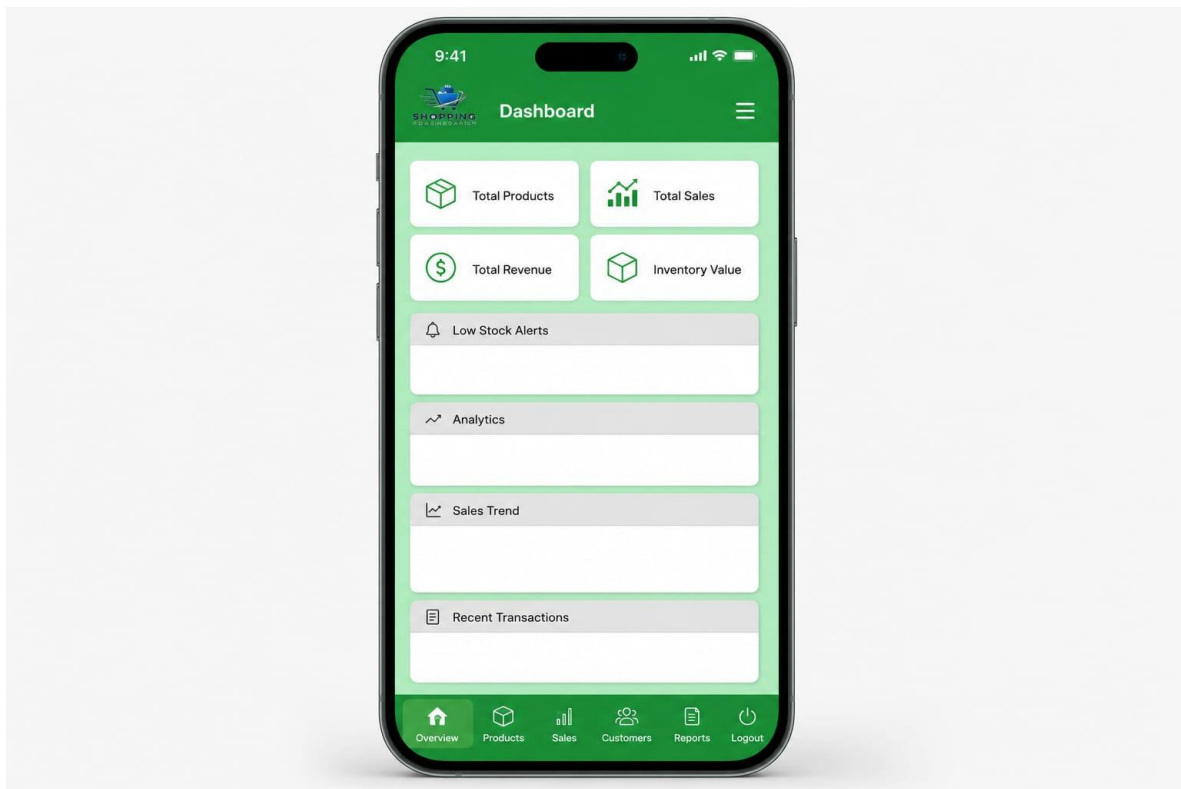
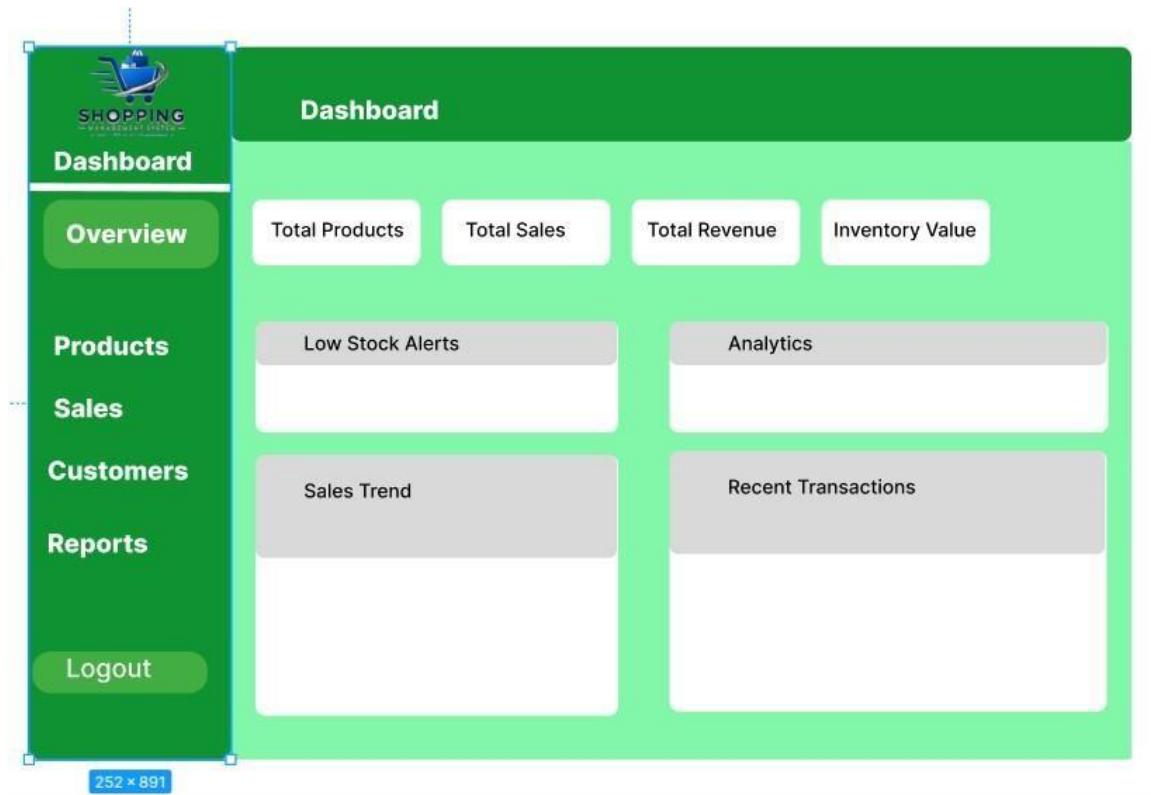


Fig 4: GUI prototype of the Smart Shop dashboard in Figma



Week 2 Reflection

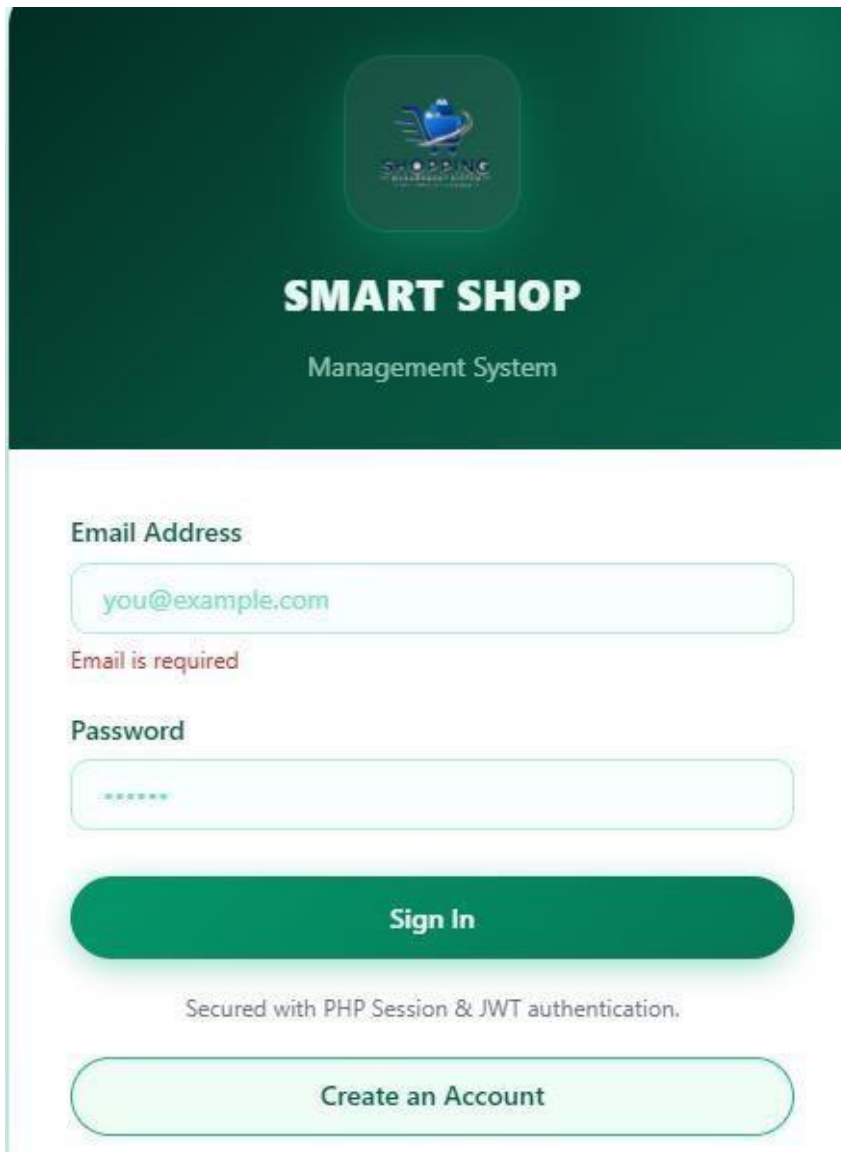
I designed wireframes for the Smart Shop Management System focusing on usability and scalability. The dashboard layout displays key metrics such as sales, products, customers, and revenue using KPI cards. A left sidebar navigation was selected to support future modules. The color palette is clean and professional, while the login page was kept minimal. Mobile responsiveness was considered early to ensure consistent usability across devices.

WEEK 3: JavaScript and PHP Basics

Frontend Interaction and Backend Foundations

Required Evidence

Fig 1: JavaScript form validation on the login page preventing empty submission

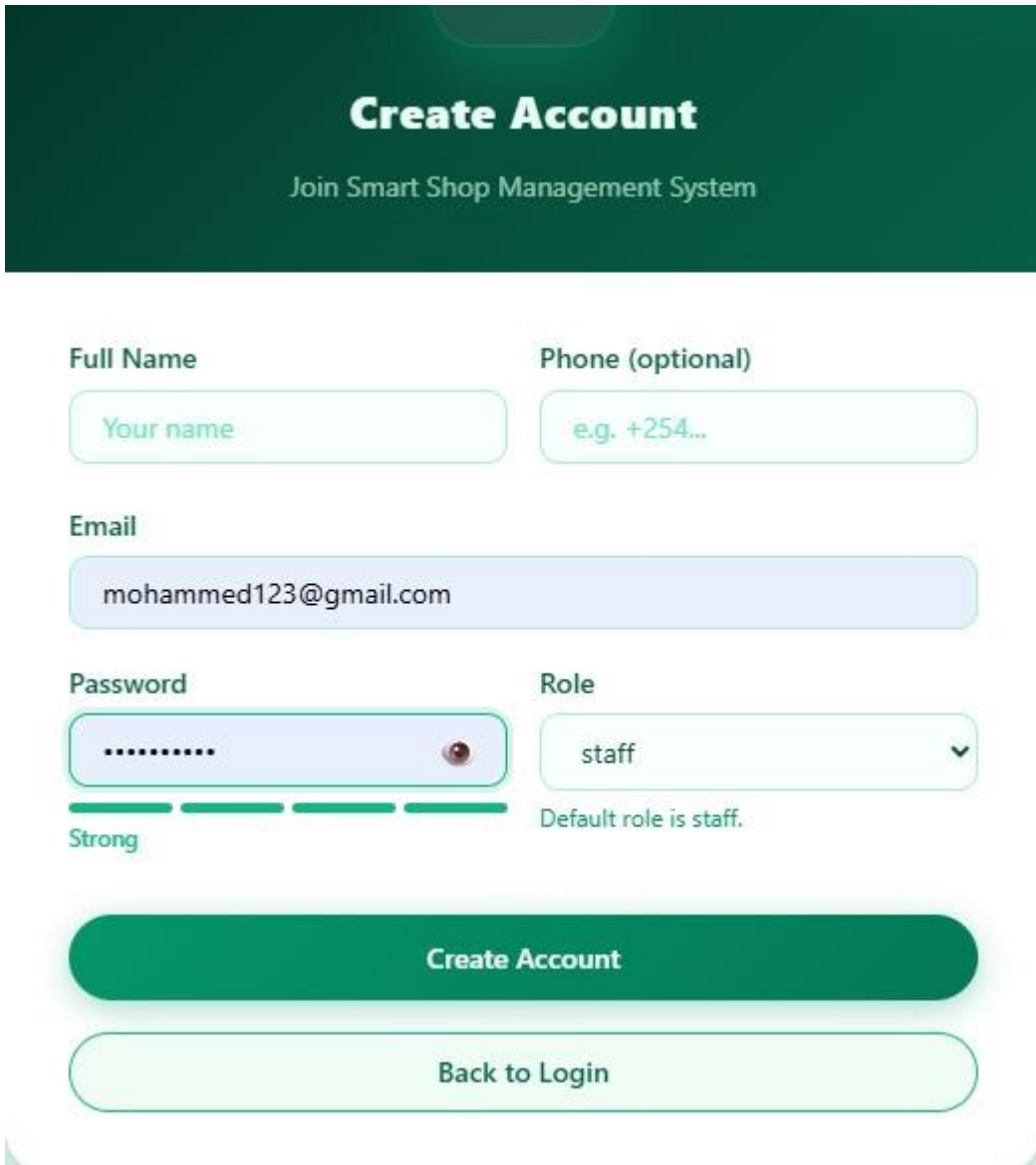


The image displays the user interface for the 'SMART SHOP Management System'. At the top, a dark green header contains a logo with a shopping cart and the text 'SMART SHOP Management System'. Below the header, the login and registration section is set against a light green background. It features two input fields: 'Email Address' with the placeholder 'you@example.com' and a red error message 'Email is required' below it; and 'Password' with a masked input '*****'. A dark green 'Sign In' button is positioned below the password field. Underneath the button, a small grey text line states 'Secured with PHP Session & JWT authentication.'. At the bottom of the section is a light green 'Create an Account' button.

Fig 2: PHP database connection script (db.php) for MySQL

```
api > config > db.php > ...
1  <?php
2
3
4  // Database Credentials
5  define('DB_HOST', '127.0.0.1');
6  define('DB_PORT', '3306');
7  define('DB_USER', 'root');
8  define('DB_PASS', '');
9  define('DB_NAME', 'shopping_system_db');
10
11  /**
12   * Connects to the MySQL database via PDO.
13   *
14   * @return PDO|null The PDO database connection instance, or null on error.
15   */
16  1 reference
17  function connectDB() {
18      static $conn = null;
19
20      if ($conn !== null) {
21          return $conn;
22      }
23
24      try {
25          $dsn = "mysql:host=" . DB_HOST . ";port=" . DB_PORT . ";dbname=" . DB_NAME;
26          $options = [
27              PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
28              PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
29              PDO::ATTR_EMULATE_PREPARES => false,
30          ];
31      } catch (PDOException $e) {
32          // Handle connection error
33      }
34      return $conn;
35  }
```

Fig 3: Password Strength Checker on the Registration Form



The image shows a 'Create Account' form for the 'Smart Shop Management System'. The form is set against a dark green background with a lighter green header. The header contains the title 'Create Account' and the subtitle 'Join Smart Shop Management System'. The form fields are arranged in a grid-like structure. The first row has two fields: 'Full Name' with a placeholder 'Your name' and 'Phone (optional)' with a placeholder 'e.g. +254...'. The second row has a single field for 'Email' with the placeholder 'mohammed123@gmail.com'. The third row has two fields: 'Password' with a masked password '.....' and a strength indicator showing four green bars and the word 'Strong', and 'Role' with a dropdown menu showing 'staff' and a note 'Default role is staff.'. At the bottom, there are two buttons: a green 'Create Account' button and a light green 'Back to Login' button.

Create Account

Join Smart Shop Management System

Full Name

Phone (optional)

Email

Password Strong

Role Default role is staff.

Create Account

Back to Login

Fig 4: PHP script returning JSON response from the products table

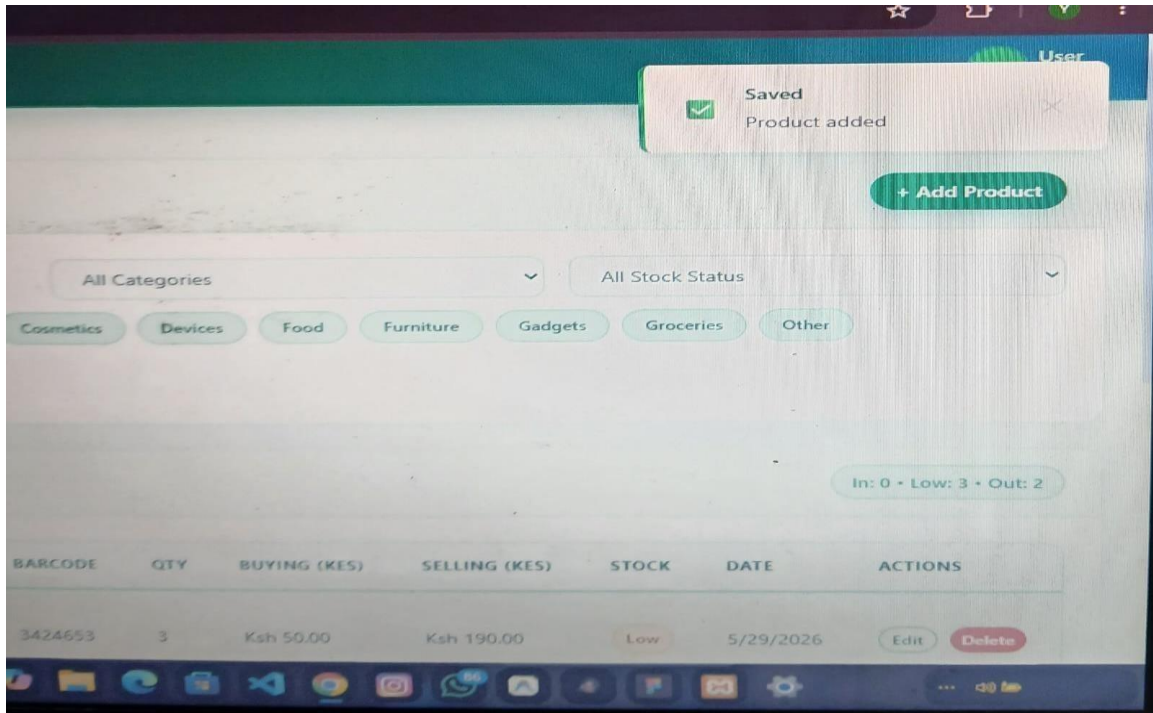
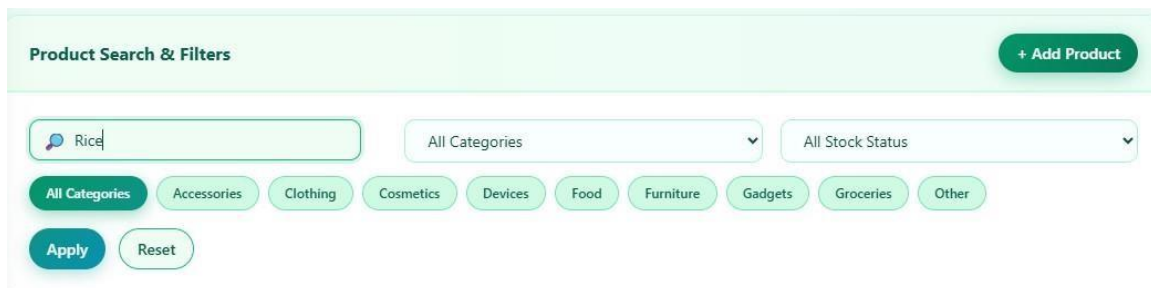


Fig 5: Dynamic user input handling — product search filtering results in real time



Week 3 Reflection

I implemented JavaScript validation on login and registration forms to prevent invalid submissions. A password strength checker improves security during registration. PHP was used to connect to the MySQL shopping_system_db and return product data as JSON. I also built a real-time product search feature using JavaScript to filter results dynamically. This week strengthened the integration between the HTML frontend and the PHP/MySQL backend.

WEEK 4: Dynamic Backend Processing & Real User Interaction

Task 1: Understanding Server-Side Programming (Practical)

Enter Your Name

Task 2: HTML Forms and PHP Integration

Login Form

Smart Shop Management System | Advanced Web Design and Development

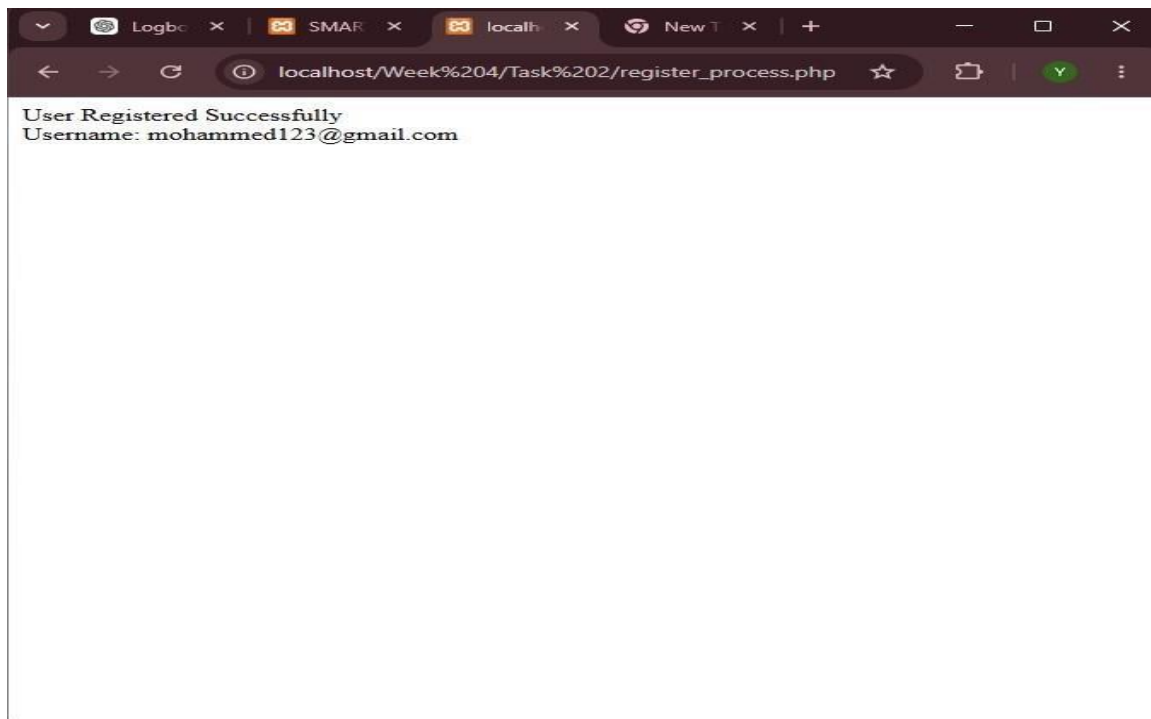


localhost/Week%204/Task%202/login.php

mohammed123@gmail.com

Login

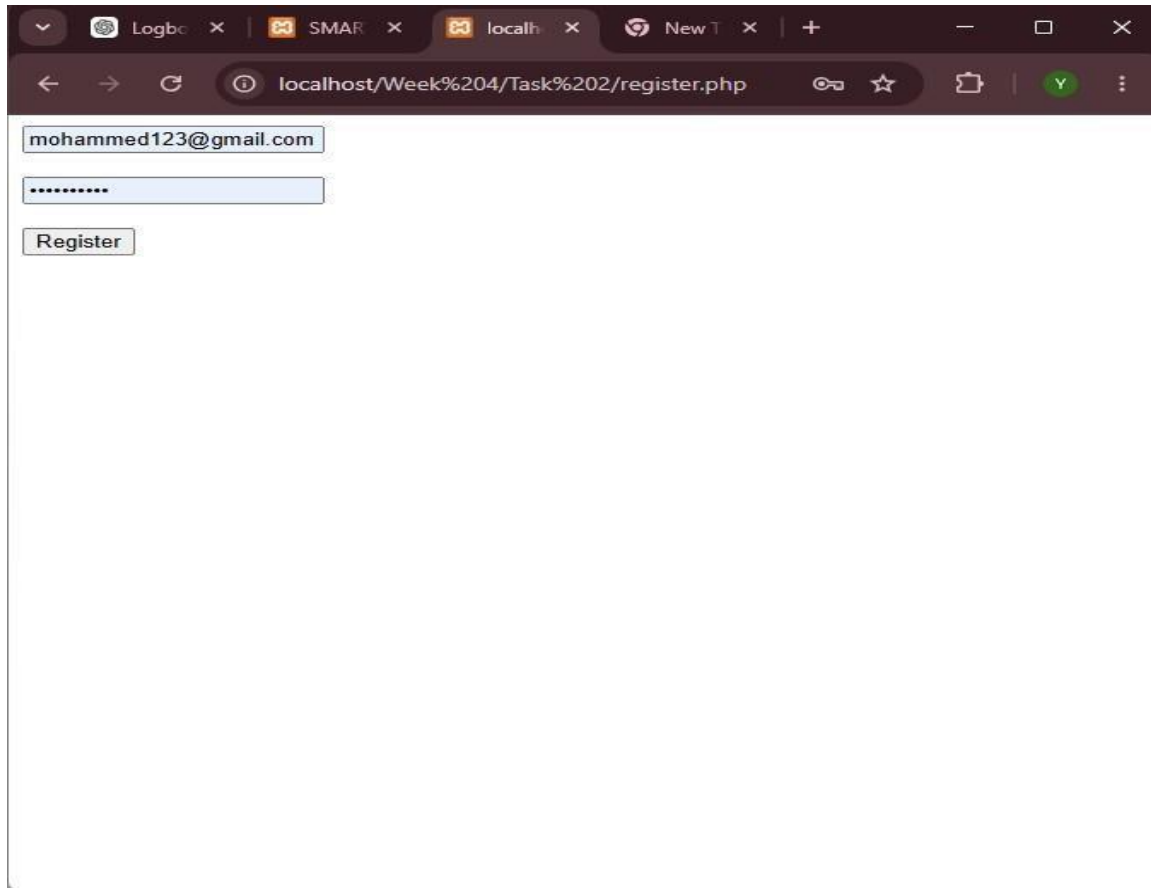
Output processing pages



localhost/Week%204/Task%202/register_process.php

User Registered Successfully
Username: mohammed123@gmail.com

Register Form



The screenshot shows a web browser window with the address bar displaying 'localhost/Week%204/Task%202/register.php'. The browser has several tabs open, including 'Logbo', 'SMAR', 'localh', and 'New T'. The registration form consists of two text input fields. The first field contains the email address 'mohammed123@gmail.com'. The second field contains a masked password represented by seven asterisks '*****'. Below these fields is a button labeled 'Register'.

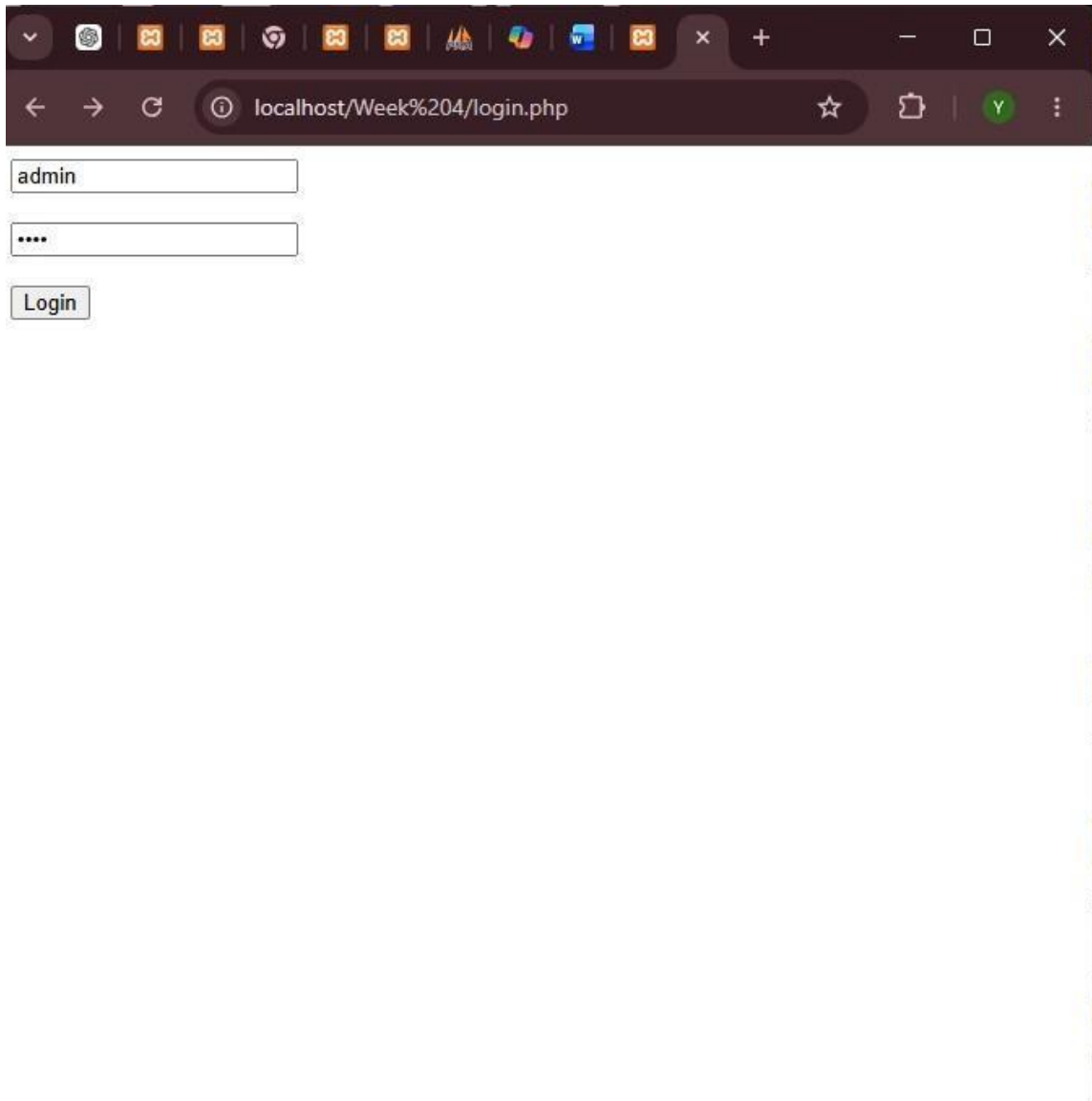
Task 3: Simple Authentication System (Sessions)

Goal

Create login logic + session-based authentication

Login Logic with Session

Successful login



Dashboard page

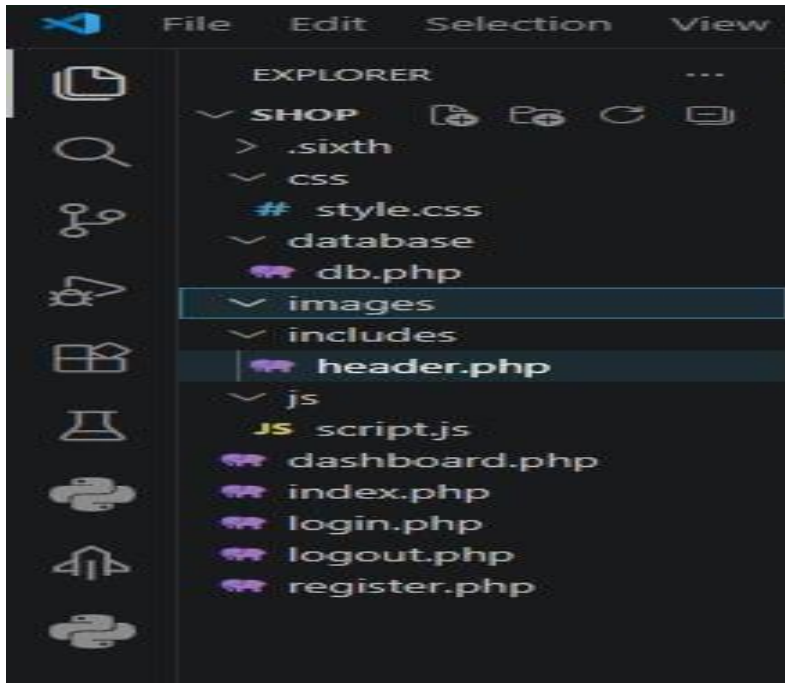


Invalid Login attempt

Invalid Login Details

Task 4: Backend Folder Organization

Professional project structure



Task 5: Modern Backend Alternatives (Theory)

Purpose of Task 5

The goal of this task is to **expose students to modern backend technologies** used in industry today and show how traditional PHP-based systems map to **more scalable, cloud-ready solutions**.

This task is **theoretical + conceptual**, not heavy coding.

Why Modern Backend Alternatives Matter

Traditional PHP systems are still widely used, but **modern applications** (e-commerce, SaaS, APIs, mobile apps) often require:

- High scalability
- Real-time communication
- API-first architecture
- Cloud deployment
- Secure token-based authentication

This is where **modern backend stacks** come in.

Comparison: PHP vs Modern Backend Technologies

Traditional PHP Concept	Modern Alternative	Explanation
PHP Backend	Node.js	Node.js allows JavaScript to run on the server, enabling full-stack JavaScript development and better performance for real-time applications.
XAMPP	Express.js	Express.js is a lightweight Node.js framework that replaces Apache/PHP by handling routing, middleware, and APIs efficiently.
MySQL	MongoDB	MongoDB is a NoSQL database that stores data in flexible JSON-like documents, making it suitable for dynamic and scalable systems.
PHP Sessions	JWT Authentication	JWT provides stateless authentication, making it ideal for APIs, mobile apps, and distributed systems.

Traditional Hosting	Cloud Deployment	Cloud platforms allow automatic scaling, continuous deployment, and global access compared to shared hosting.
---------------------	------------------	---

Conceptual Mapping

PHP → Node.js

- PHP executes per request
- Node.js uses an **event-driven, non-blocking architecture**
- Better for chat systems, dashboards, APIs

Sessions → JWT

- PHP sessions are stored on the server
- JWT stores authentication data inside a signed token
- No server-side session storage required

MySQL → MongoDB

- MySQL uses tables and fixed schemas
- MongoDB uses collections and flexible documents
- Ideal for products, users, and logs that change often

Real-World Usage Examples

These concepts are used in:

- **E-commerce systems**
- **SaaS platforms**
- **ERP systems**
- **Mobile applications**
- **REST APIs**

Frameworks that use these ideas:

- Laravel (PHP)
- Django (Python)
- Node.js + Express
- ASP.NET
- Spring Boot

Week 4 Reflection

This week introduced backend processing using PHP and real user interaction. I implemented form handling using POST methods and created dynamic pages that process user input. A simple authentication system using PHP sessions was developed to control access to the dashboard. I also organized the project into a professional folder structure to improve maintainability. These tasks strengthened my understanding of the request–response cycle and laid the foundation for secure web applications.

WEEK 5: Database Components & CRUD Operations

Task 1: Introduction to Databases (Concept + Understanding)

A database is used to store, retrieve, update, and manage data permanently.

Why Databases Are Important

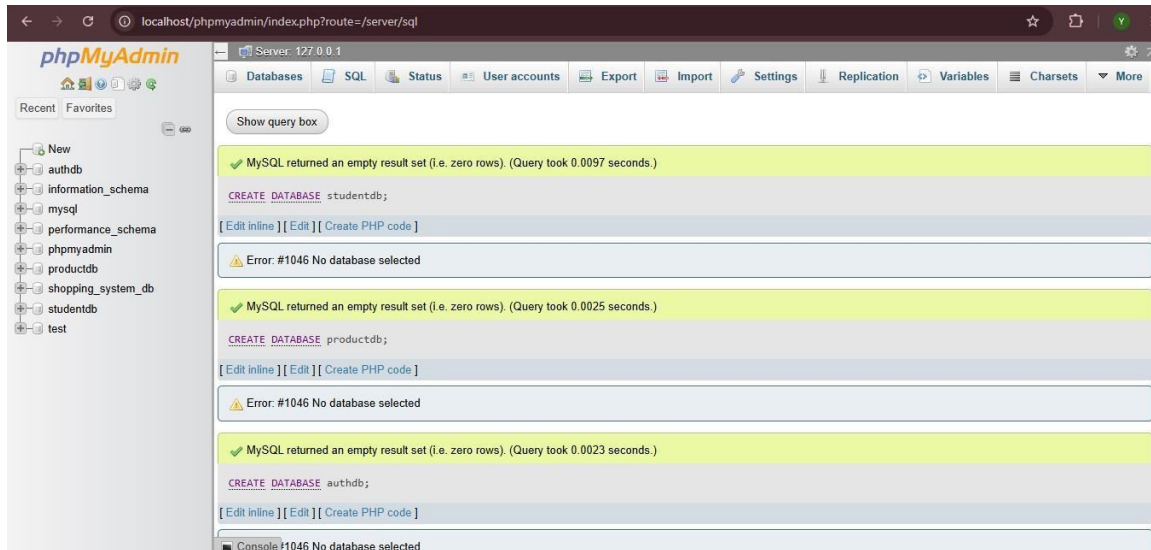
- Store large amounts of data
- Ensure data consistency
- Allow multiple users
- Support real-time systems

Database Types

Relational Databases	NoSQL Databases
MySQL	MongoDB
PostgreSQL	Firebase
SQL Server	CouchDB

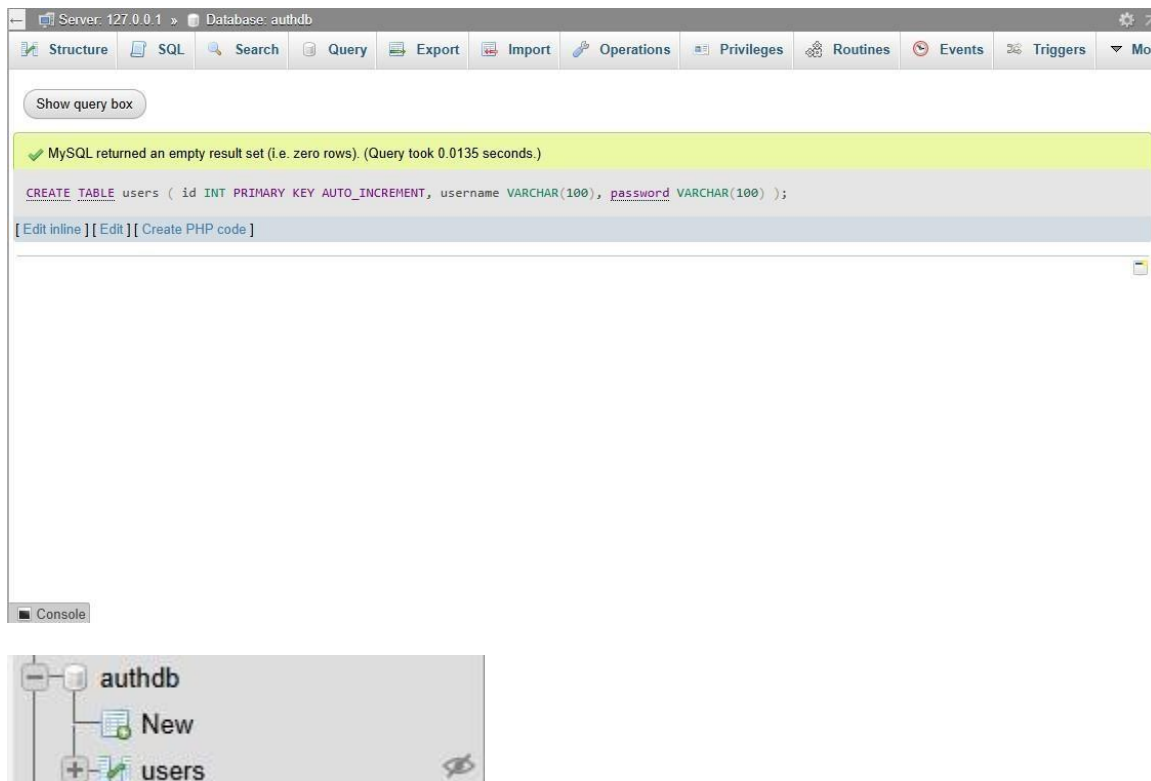
Task 2: Database Creation (Practical)

Creating databases using phpMyAdmin



Task 3: Creating Tables

Creating tables inside the database



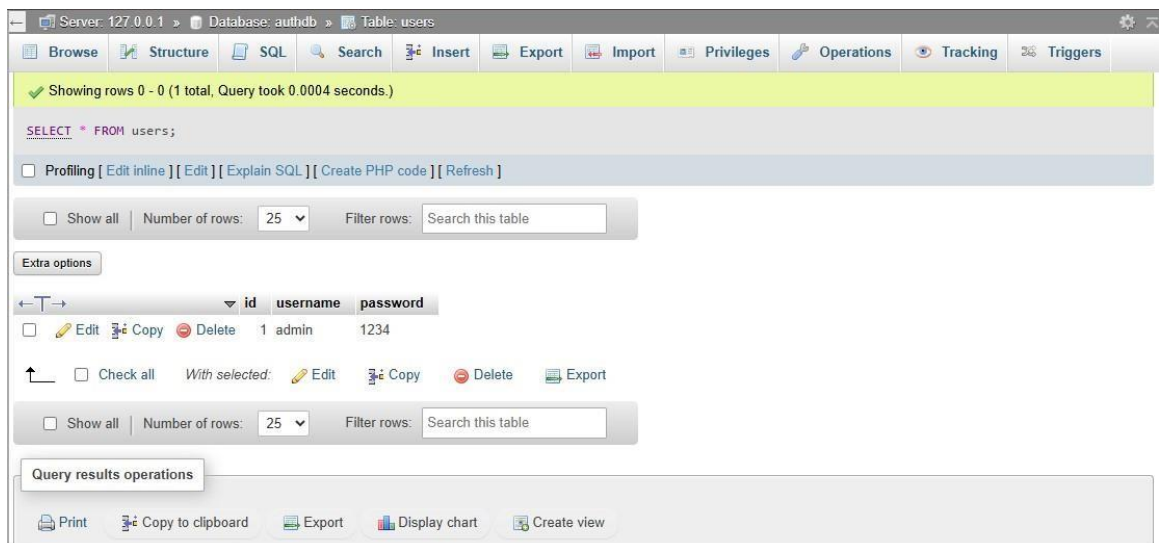
Task 4: CRUD Operations

CRUD = **C**reate, **R**ead, **U**ppdate, **D**elete

CREATE (Insert Record)



READ (View Records)



UPDATE (Edit Record)

Smart Shop Management System | Advanced Web Design and Development

The screenshot shows the phpMyAdmin interface for the 'users' table in the 'authdb' database. The top navigation bar includes tabs for Browse, Structure, SQL, Search, Insert, Export, Import, Privileges, Operations, Tracking, and Triggers. A 'Show query box' button is visible. The main area displays a green success message: '1 row affected. (Query took 0.0003 seconds.)'. Below this, the executed SQL query is shown: `UPDATE users SET password = 'admin123' WHERE username = 'admin';`. There are links for '[Edit inline]', '[Edit]', and '[Create PHP code]'. A second green message states: 'Showing rows 0 - 0 (1 total, Query took 0.0003 seconds.)'. The SQL query shown is: `SELECT * FROM `users` WHERE 1;`. Below the query, there are links for '[Profiling]', '[Edit inline]', '[Edit]', '[Explain SQL]', '[Create PHP code]', and '[Refresh]'. A control bar shows 'Show all', 'Number of rows: 25', and a 'Filter rows: Search this table' input. An 'Extra options' button is also present. At the bottom, a table view shows the 'users' table with columns 'id', 'username', and 'password'. The table contains one row with 'id' 1, 'username' 'admin', and 'password' 'admin123'. Action buttons like 'Edit', 'Copy', 'Delete', 'Check all', 'With selected', 'Export', and 'Delete' are available for the table data.

DELETE (Delete Record)

The screenshot shows the phpMyAdmin interface for the 'users' table. The top navigation bar includes tabs for Structure, SQL, Search, Query, Export, Import, Operations, Privileges, Routines, Events, Triggers, and More. A 'Show query box' button is visible. The main area displays a green success message: '1 row deleted. (Query took 0.0004 seconds.)'. Below this, the executed SQL query is shown: `DELETE FROM users WHERE username = 'admin';`. There are links for '[Edit inline]', '[Edit]', and '[Create PHP code]'. The rest of the interface is empty.

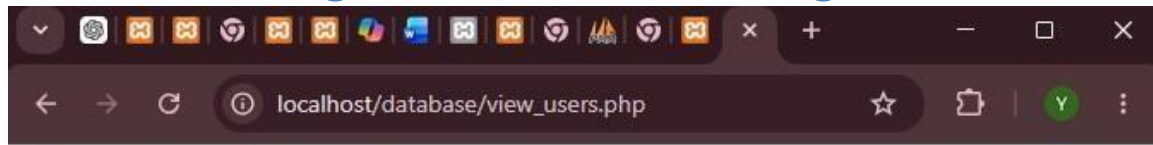
Task 5: Connecting PHP to Database

Success Connection

Smart Shop Management System | Advanced Web Design and Development



Task 6: Fetching Database records using PHP



Database Connected Successfully

Introduction to Modern Backend Alternatives

As web applications grow in scale and complexity, modern backend technologies are increasingly used to overcome the limitations of traditional PHP-based systems. This section introduces modern alternatives and explains how they relate to traditional tools.

Comparison of Traditional and Modern Technologies

Traditional Technology	Modern Equivalent	Explanation
MySQL	MongoDB	MySQL stores data in structured tables with fixed schemas, while MongoDB stores data in flexible JSON like documents, making it more scalable for modern applications.
phpMyAdmin	MongoDB Compass	phpMyAdmin is a web interface for managing MySQL databases, while MongoDB Compass provides a graphical interface for managing MongoDB collections and documents.
SQL Queries	Mongoose ORM	SQL uses structured queries to interact with databases, while Mongoose ORM allows developers to interact with MongoDB using JavaScript objects and models.
PHP Backend	Express.js API	PHP handles backend logic through server-side scripts, whereas Express.js is a Node.js framework used to build fast, scalable RESTful APIs.

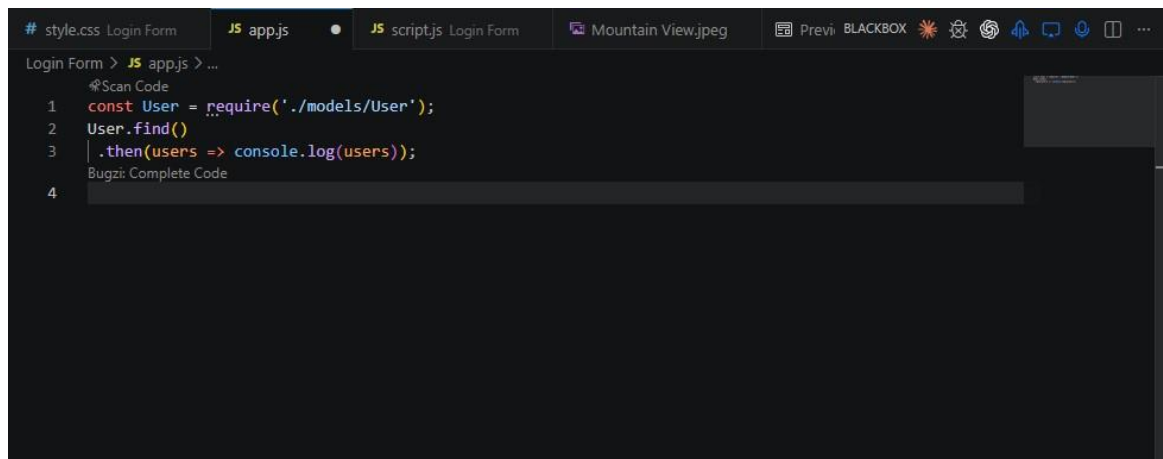
Understanding the MEVN Stack

The **MEVN stack** is a modern full-stack development approach that consists of:

- **MongoDB** – Database
- **Express.js** – Backend framework
- **Vue.js** – Frontend framework
- **Node.js** – Server runtime

This stack enables developers to use **JavaScript across the entire application**, from frontend to backend, improving development efficiency and performance.

MEVN Equivalent Example (Explanation Only)



```
# style.css Login Form JS app.js JS script.js Login Form Mountain View.jpeg Prev BLACKBOX
Login Form > JS app.js > ...
1 const User = require('./models/User');
2 User.find()
3 .then(users => console.log(users));
4
```

Explanation of the Code (Theory)

- `require('./models/User')` imports a **User model**, which represents a database collection.
- `User.find()` retrieves **all user records** from the MongoDB database.

Week 5 Logbook Reflection

This week focused on database integration and CRUD operations using MySQL and PHP. I created databases and tables to store user data and implemented Create, Read, Update, and Delete operations using SQL queries. PHP was connected to the database using `mysqli`, allowing dynamic retrieval of records. I also integrated the login system with the database for authentication. This week enhanced my understanding of backend-data integration and building data-driven web applications.